

Sistemas Operacionais

Alocação de memória

Professor Romário da Silva Borges

Alocadores de memória

Alocar memória

Alocador de memória física: organiza a memória física do computador, definindo as áreas que podem ser alocadas pelo núcleo e processos e as áreas reservadas (BIOS, DMA, etc.). Gerencia o uso das áreas alocáveis, atendendo requisições de alocação/liberação do núcleo e de processos.

Alocador de espaço de núcleo: o núcleo do SO continuamente cria e destrói muitas estruturas de dados relativamente pequenas, como descritores de arquivos abertos, de processos, sockets de rede, pipes, etc. O alocador de núcleo obtém áreas de memória do alocador físico e as utiliza para alocar essas estruturas para o núcleo.

Espaço de endereçamento virtual de um processo

Alocador de espaço de usuário: um processo pode solicitar blocos de memória para armazenar estruturas de dados dinâmicas, através de operações como malloc e free. O alocador de memória do processo geralmente é implementado por bibliotecas providas pelo sistema operacional, como a LibC. Essas bibliotecas interagem com o núcleo para solicitar o redimensionamento da seção HEAP do processo quando necessário.

Os Três Níveis de Alocação do Sistema.

Nível 3

Alocador de Espaço de Usuário (Aplicação/Topo)

Onde roda a LibC e o malloc(). Roda dentro do processo, gerenciando o Heap e pedindo blocos maiores ao S0 apenas quando necessário.

Nível 2

Alocador de Núcleo (S0)

Gerencia a memória interna do próprio S0 para manter o sistema rodando de forma hiper-eficiente.

Nível 1

Alocador de Memória Física (Hardware/Base)

Gerencia os quadros físicos (frames) reais na placa-mãe.

Diferentes clientes exigem diferentes garçons. O S0 possui camadas especializadas para não sobrecarregar o hardware com pequenos pedidos.

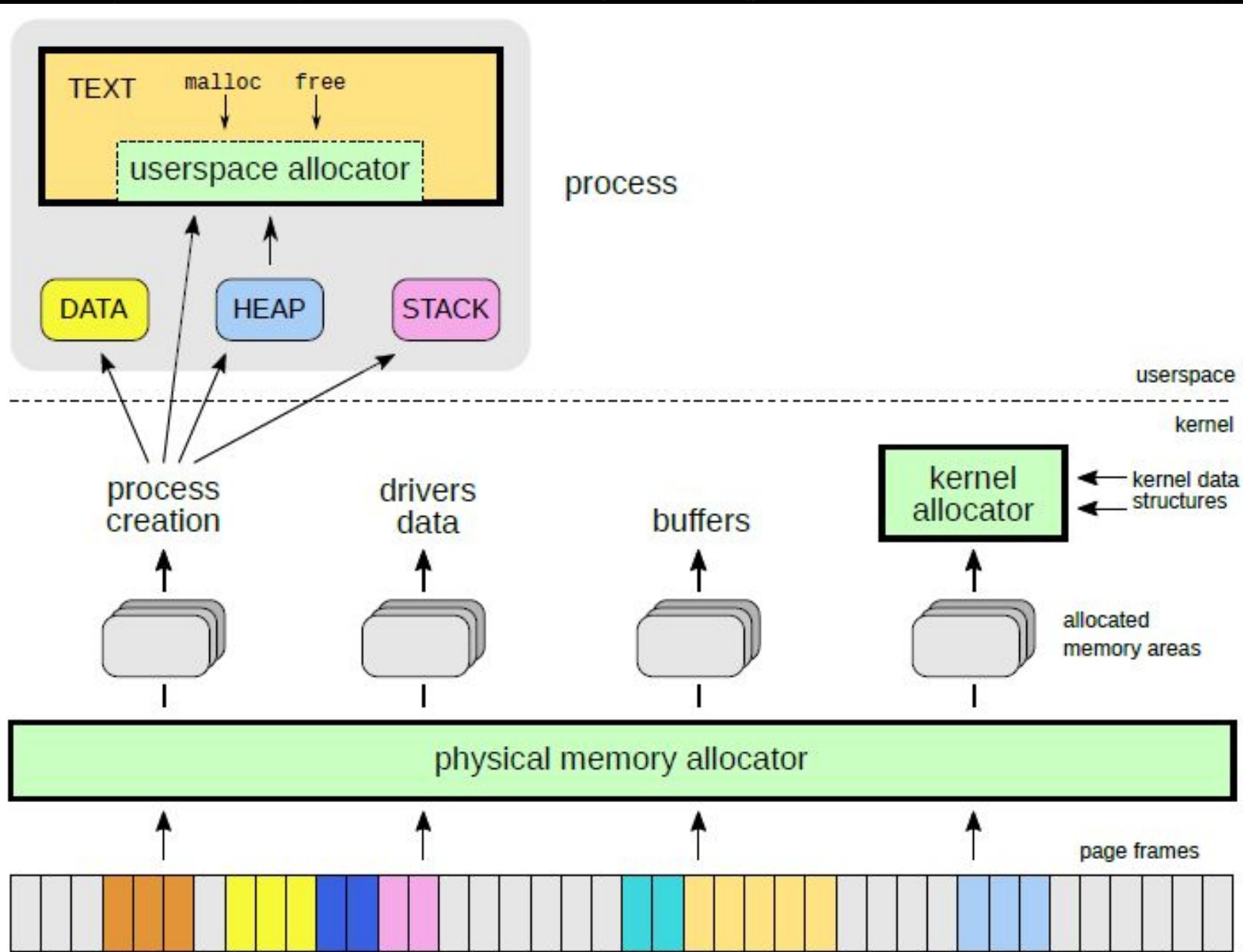


Figura 16.1: Mecanismos de alocação de memória.

Fragmentação

O desafio da alocação

O problema básico de alocação consiste em manter uma grande área de memória RAM e atender um fluxo de requisições de alocação e liberação de partes dessa área para o sistema operacional ou para as aplicações. Essas requisições ocorrem o tempo todo, em função das tarefas em execução no sistema, e devem ser atendidas rapidamente.

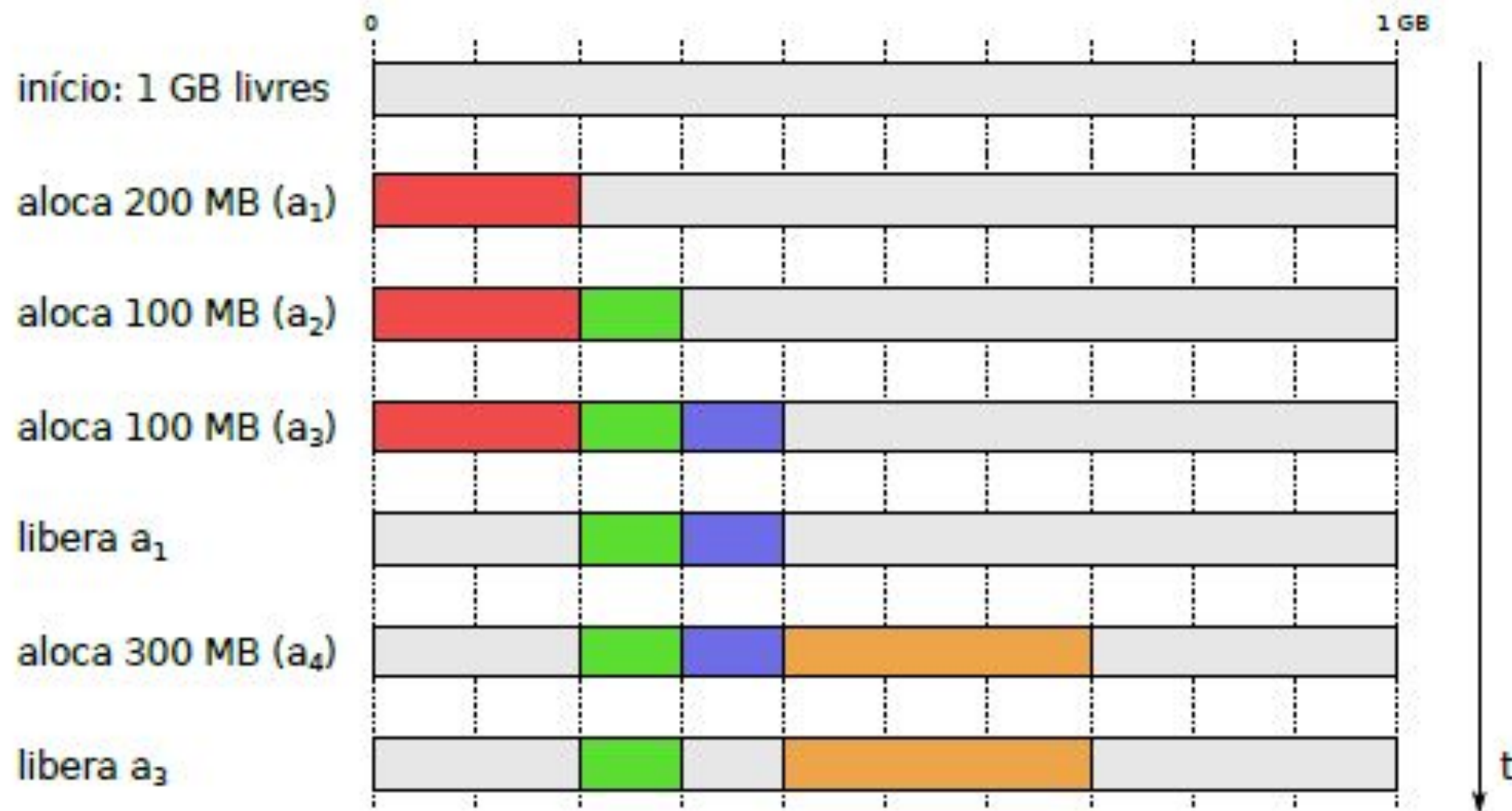


Figura 16.2: Sequência de alocações e liberações de memória.

Fragmentação externa

Ao longo da vida de um sistema, áreas de memória são alocadas e liberadas continuamente. Com isso, podem surgir áreas livres (“buracos” na memória) entre as áreas alocadas. Na Figura anterior pode-se observar que o sistema ainda tem 600 MB de memória livre após a sequência de operações, mas somente requisições de alocação de até 300 MB pode ser aceitas, pois esse é o tamanho da maior área livre contínua disponível. Esse fenômeno se chama **fragmentação externa**, pois fragmenta a memória livre, fora das áreas alocadas.

Fragmentação

A **fragmentação externa** é muito prejudicial, porque limita a capacidade de alocação de memória do sistema. Além disso, quanto mais fragmentada estiver a memória livre, maior o esforço necessário para gerenciá-la, pois mais longas serão as listas encadeadas de área de memória livres. Pode-se enfrentar o problema da fragmentação externa de duas formas: minimizando sua ocorrência, através de estratégias de alocação, desfragmentando periodicamente a memória do sistema, ou permitindo a fragmentação interna.

Estratégias de alocação

Algoritmos de alocação

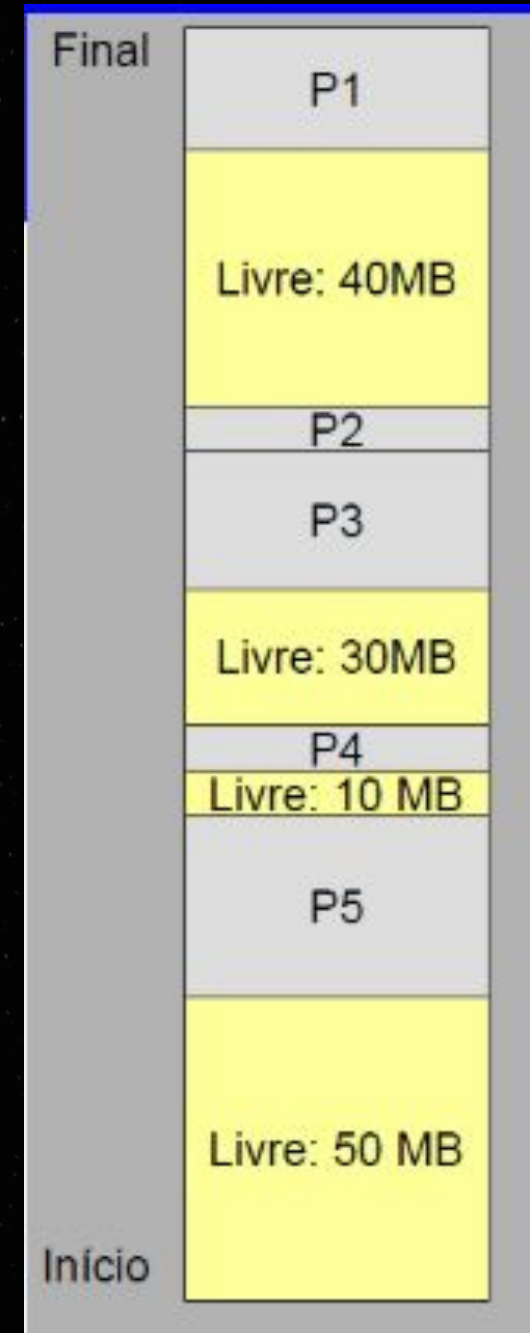
Quando o sistema operacional precisa carregar um processo na memória, ele deve encontrar um espaço livre (partição) suficientemente grande para armazená-lo.

Os algoritmos a seguir, foram muito usados em sistemas antigos e servem ainda como base conceitual para entender os sistemas modernos de gerência de heap e alocação dinâmica.

First-Fit (primeiro encaixe)

Consiste em escolher a primeira área livre que satisfaça o pedido de alocação; tem como vantagem a rapidez, sobretudo se a lista de áreas livres for muito longa.

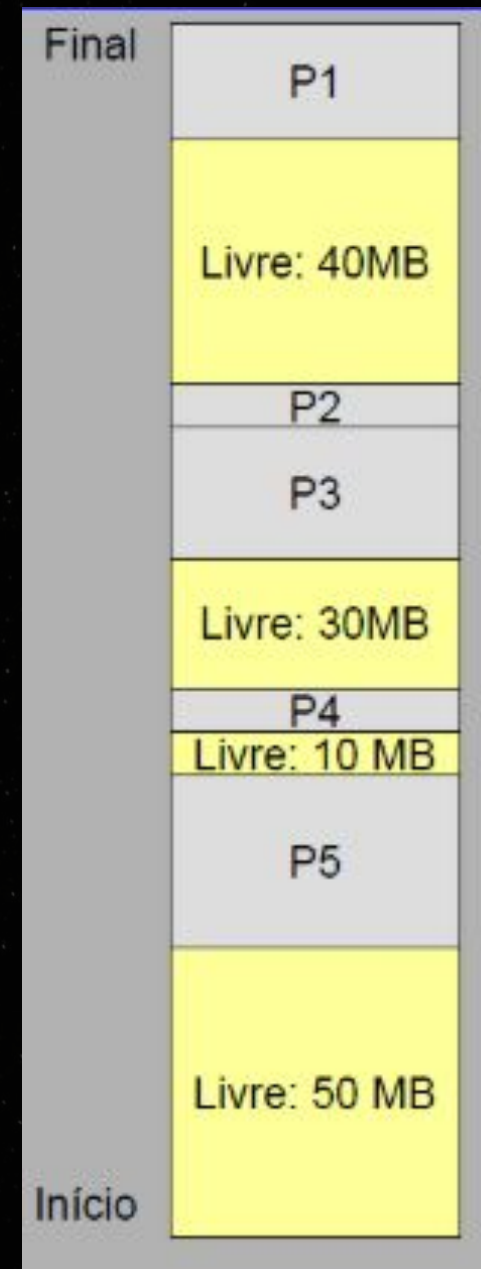
Suponha um processo de tamanho 10MB, onde ele será alocado?



Next-Fit (próximo encaixe)

Variante da estratégia first-fit que consiste em percorrer a lista de áreas a partir da última área alocada ou liberada, para que o uso das áreas livres seja distribuído de forma mais homogênea no espaço de memória.

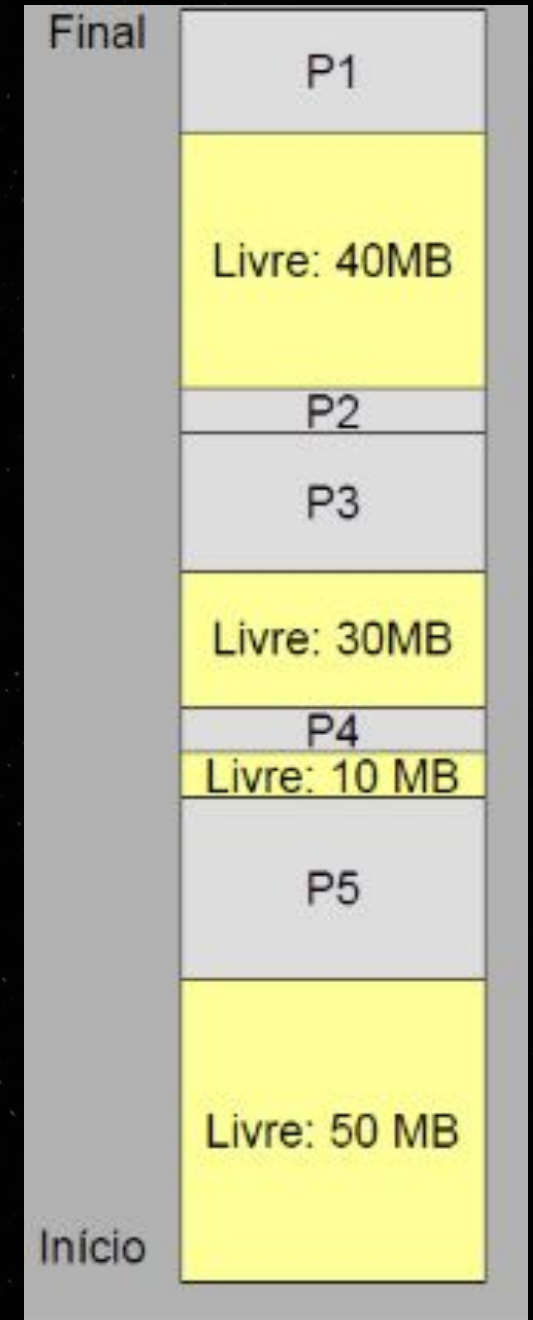
Suponha um processo de tamanho 35MB. O último processo alocado foi o P4. onde ele será alocado?



Best-Fit (melhor encaixe)

Consiste em escolher a menor área possível que possa receber a alocação, minimizando o desperdício de memória. Contudo, algumas áreas livres podem ficar pequenas demais e com isso se tornarem inúteis.

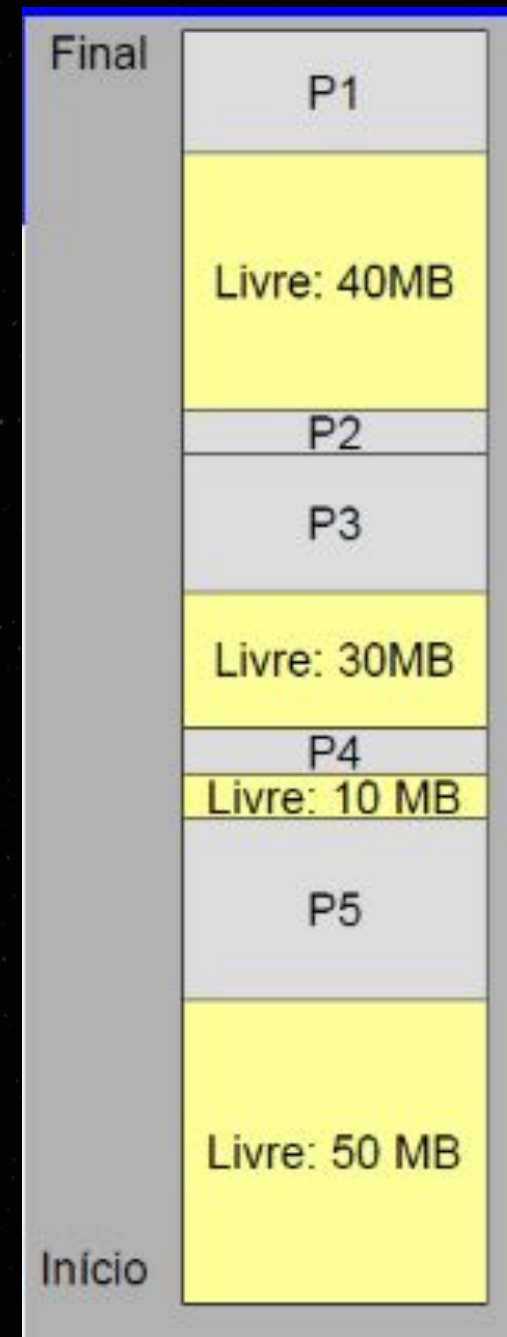
Suponha um processo de tamanho 28MB, onde ele será alocado?



Worst-Fit (pior encaixe)

Consiste em escolher sempre a maior área livre possível, de forma que a “sobra” seja grande o suficiente para ser usada em outras alocações.

Suponha um processo de tamanho 3MB, onde ele será alocado?



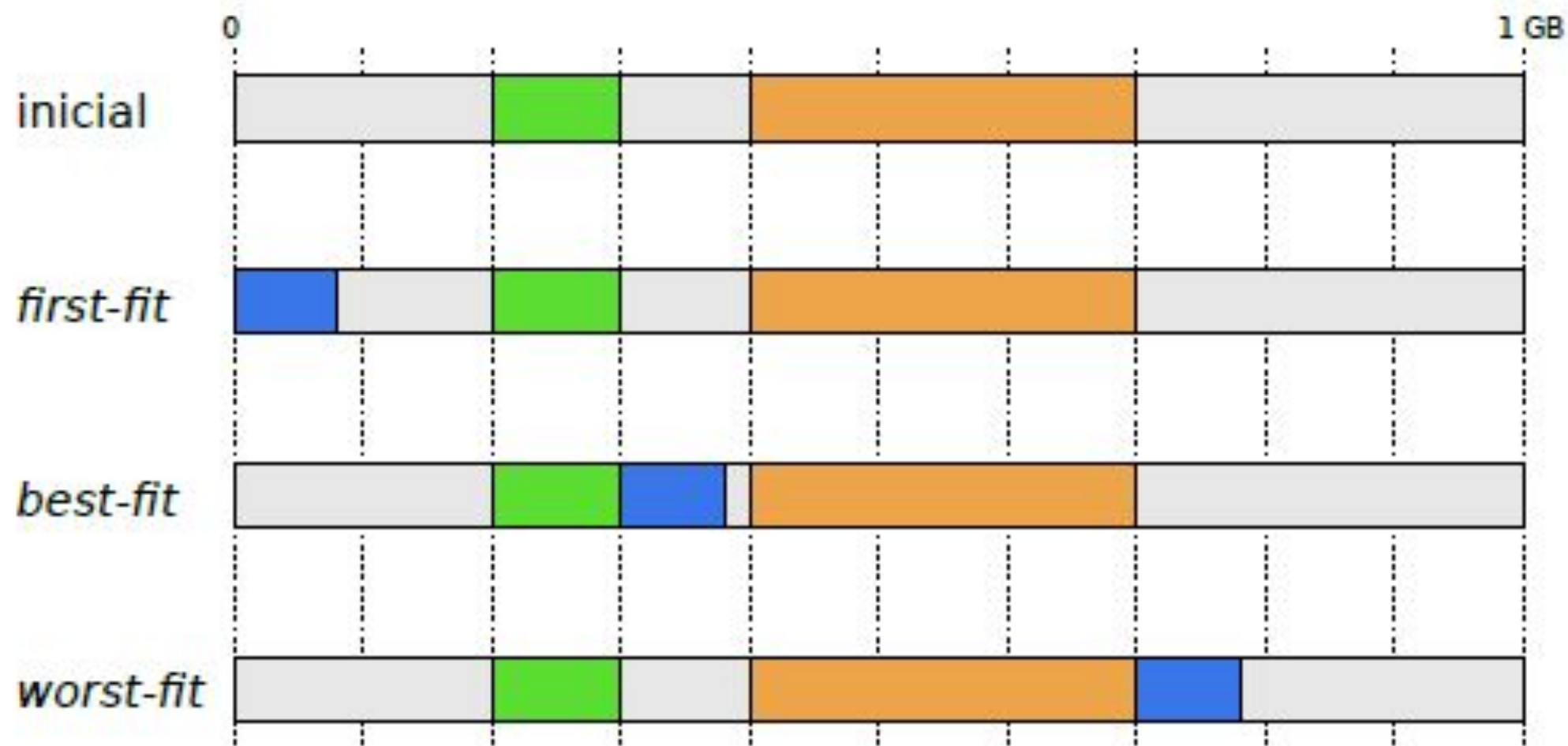


Figura 16.3: Estratégias para minimizar a fragmentação externa.

Qual algoritmo é o melhor?

Desfragmentação

Desfragmentação

Outra forma de tratar a fragmentação externa consiste em desfragmentar a memória periodicamente. Para tal, as áreas de memória usadas pelos processos devem ser movidas na memória de forma a concatenar as áreas livres e assim diminuir a fragmentação.

Ao mover um processo na memória, suas informações de endereçamento virtual (registrador base/limite, tabela de segmentos ou de páginas) devem ser devidamente ajustadas para refletir a nova posição do processo na memória RAM. Por essa razão, a desfragmentação só pode ser aplicada a áreas de memória físicas, pois as mudanças de endereço das áreas de memória serão ocultadas pelo hardware. Ela não pode ser aplicada, por exemplo, para a gestão da seção heap de um processo.

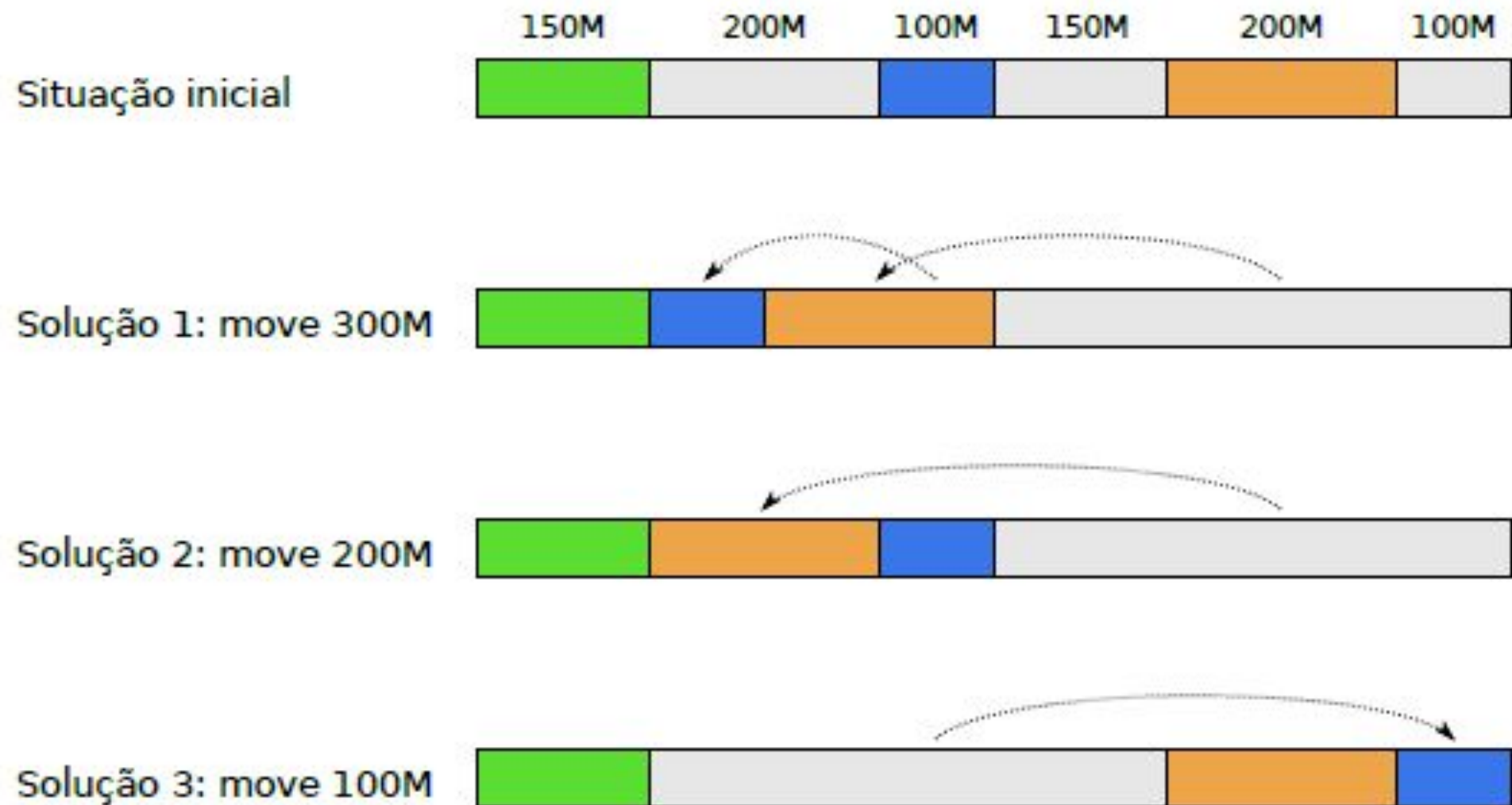


Figura 16.4: Possibilidades de desfragmentação.

Fragmentação interna

Desfragmentação

Uma alternativa para minimizar o impacto da fragmentação externa consiste em arredondar algumas requisições de alocação, para evitar sobras muito pequenas. Por exemplo, na alocação com best-fit do exemplo anterior, a área alocada poderia ser arredondada de 80 MB para 100 MB, evitando a sobra da área de 20 MB. Dessa forma, evita-se a geração de um fragmento de memória livre, mas a memória adicional alocada provavelmente não será usada por quem a requisitou. Esse desperdício de memória dentro da área alocada é denominado **fragmentação interna** (ao contrário da fragmentação externa, que ocorre nas áreas livres).

alocar 80 MB
(frag. externa)

alocar 100 MB
(frag. interna)

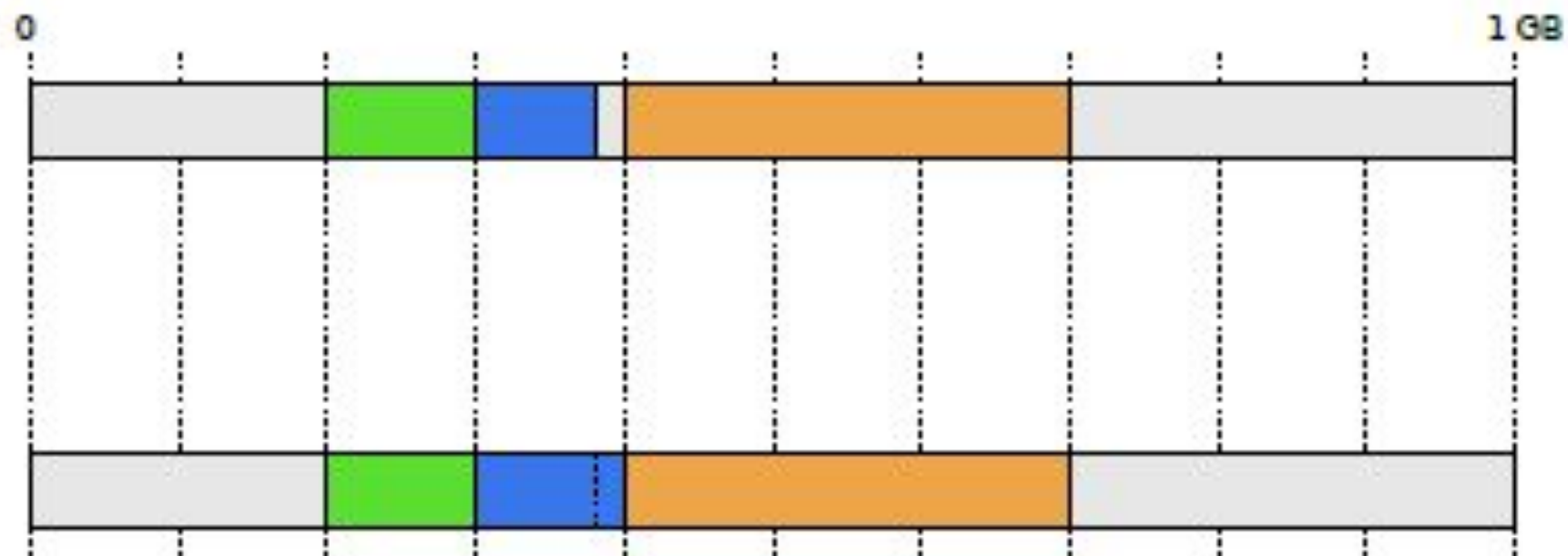


Figura 16.5: Fragmentação interna.

Matriz Diagnóstica: Formas de Desperdício

Fragmentação Externa



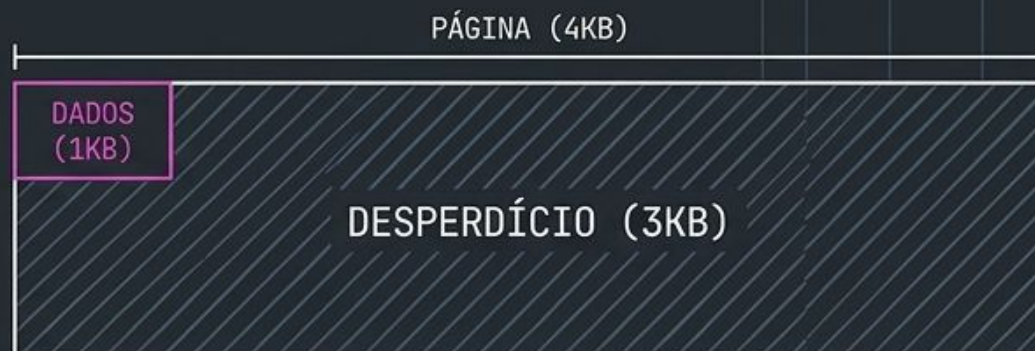
Sintoma:

Temos 80MB livres no total, mas não conseguimos alocar um bloco contínuo de 50MB.

Causa:

O espaço livre está espalhado pelo sistema. Acomete alocadores contíguos.

Fragmentação Interna



Sintoma:

O programa pediu 1KB, mas o hardware só entrega páginas inteiras de 4KB. 3KB evaporam.

Causa:

A granularidade mínima de entrega do SO é maior do que o tamanho requisitado pela aplicação.

Outras estratégias de alocação

Alocador Buddy

Existem estratégias de alocação mais sofisticadas e eficientes que as apresentadas anteriormente. Um algoritmo de alocação muito conhecido é o chamado **Buddy Allocator**, ou alocador por pares.

O alocador Buddy é usado em vários sistemas. Por exemplo, no núcleo Linux ele é usado para a alocação de memória física (page frames), entregando áreas de memória RAM para a criação de processos, para o alocador de objetos do núcleo e para outros subsistemas.

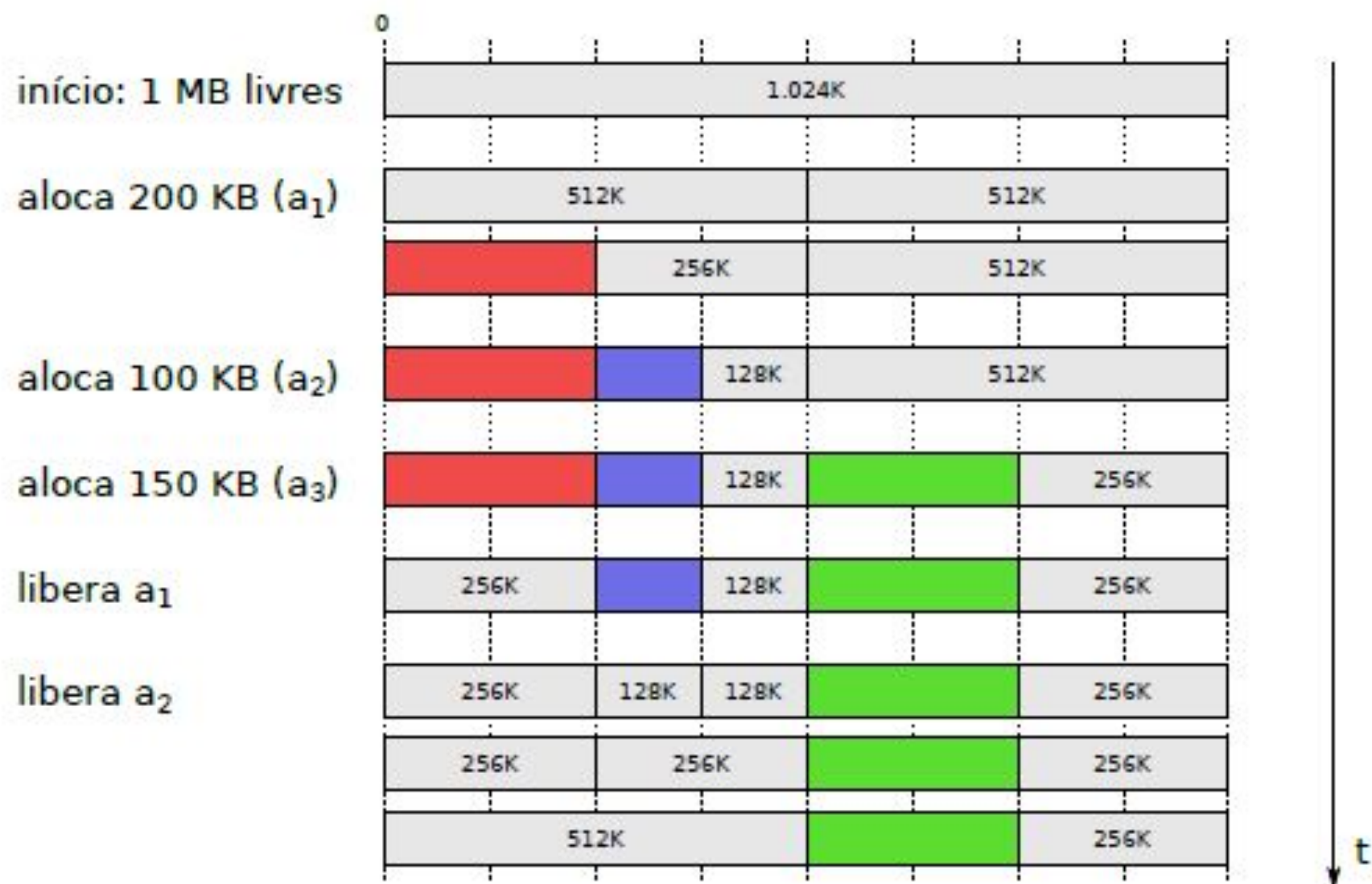


Figura 16.6: O alocador Buddy binário.

Alocador Slab

A ideia central do Slab é: em vez de o núcleo ficar pedindo e liberando memória toda vez que precisa criar uma estrutura, ele mantém reservas de objetos prontos para serem reutilizados.

O Slab funciona como um estoque organizado de objetos do kernel. Cada **cache** armazena um determinado tipo de objeto; os caches possuem slabs; e cada slab contém vários espaços para objetos daquele tipo.

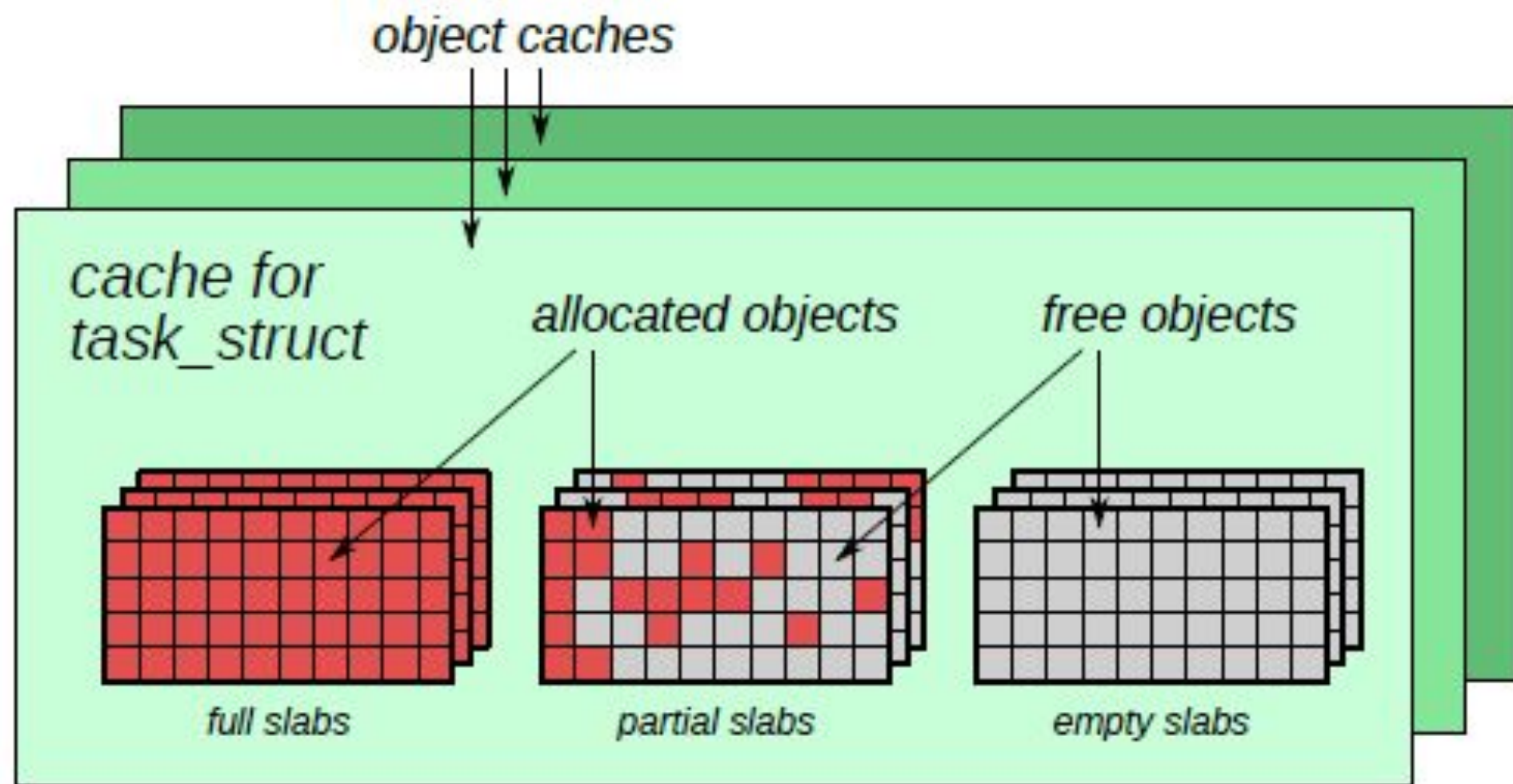
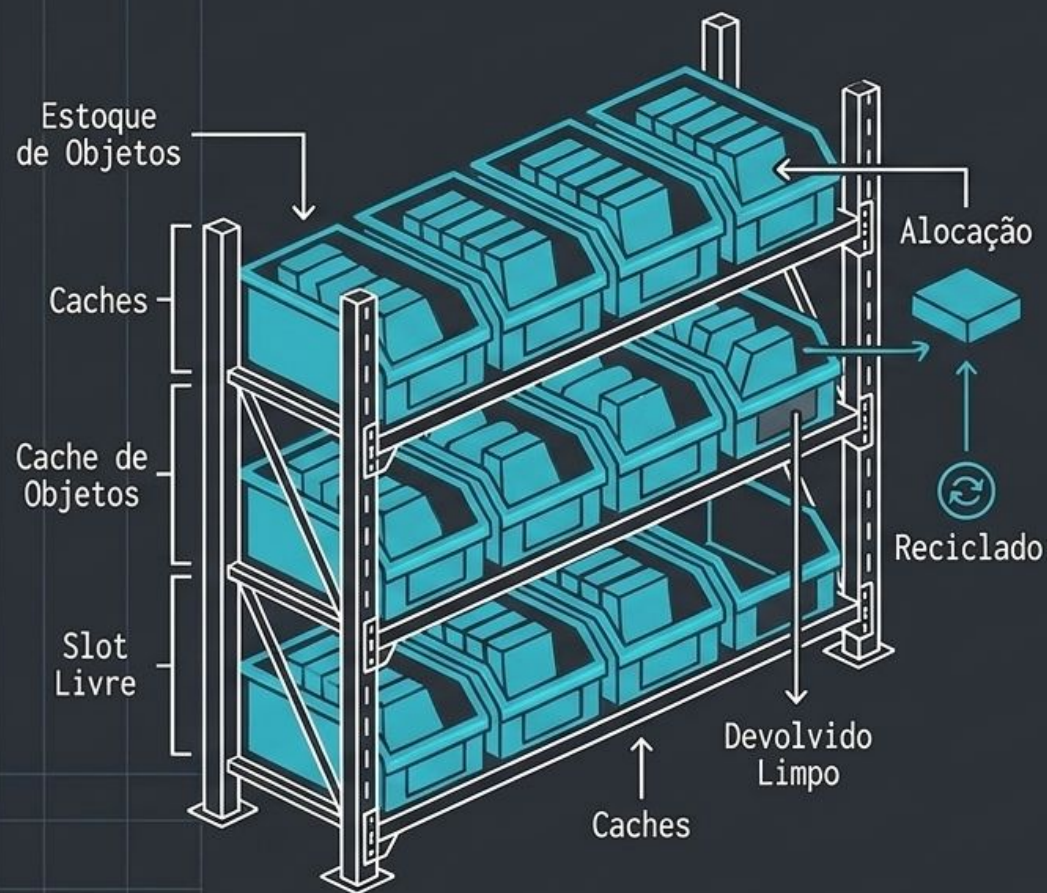


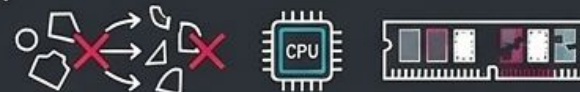
Figura 16.7: Estrutura de caches, slabs e objetos do alocador Slab.

Alocador Slab: Reciclagem de Alta Performance no Kernel.



O Problema:

O Núcleo cria e destrói milhares de pequenos objetos estruturados por segundo. Criá-los do zero e alocá-los via Buddy queima muita CPU e gera alta fragmentação interna.



A Filosofia Slab:

O SO cria prateleiras (Caches) de objetos pré-fabricados de tamanhos exatos.



A Mágica da Reciclagem:

Quando o SO não precisa mais de um objeto, ele não o destrói nem devolve a memória ao sistema global; ele apenas o devolve limpo para a prateleira, pronto para ser reciclado no próximo microssegundo.



Alocação no espaço de usuário

Da mesma forma que o núcleo, aplicações no espaço de usuário podem ter necessidade de alocar memória durante a execução para armazenar estruturas de dados dinâmicas. Ao ser criado, cada processo recebe uma área para alocação dinâmica de variáveis, chamada HEAP. O tamanho dessa seção pode ser ajustado através de chamadas de sistema que modifiquem o ponteiro *Program Break*.

Referências bibliográficas

TANENBAUM, Andrew S.; BOS, Herbert.

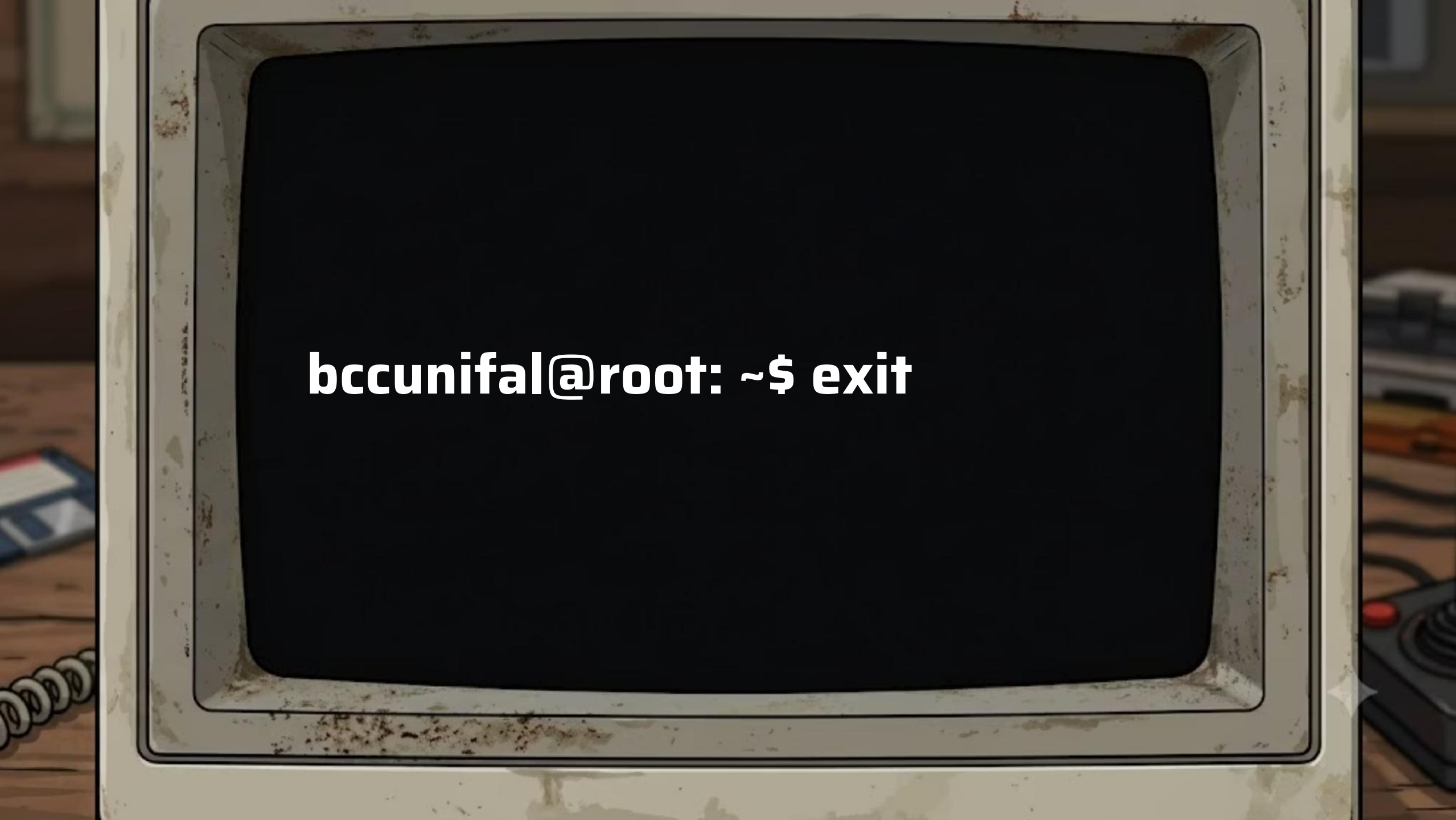
Sistemas Operacionais Modernos. 4. ed. São Paulo: Pearson, 2016.

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg.

Fundamentos de Sistemas Operacionais. 9. ed. Rio de Janeiro: LTC, 2015.

MAZIERO, Carlos A.

Sistemas Operacionais: Conceitos e Mecanismos. Curitiba: UFPR, 2019.

A terminal window is displayed on a computer monitor. The monitor has a light-colored, slightly textured bezel. The terminal window itself is a dark rectangle with a thin white border. Inside the terminal, the text 'bccunifal@root: ~\$ exit' is written in a white, monospaced font. The background of the entire image shows a desk with a keyboard and some papers, though they are out of focus.

bccunifal@root: ~\$ exit